

Natural Language Processing

Alejandro Veloz

Outline

- Transformers and GPT models
 - Training our own GPT model
 - Open source tools - FLAIR

Gensim example with OMIM data

```
['102370', 'long', 'eyelashes'],  
['102370', 'well-defined', 'eyebrows'],  
['102370', 'voice'],  
['102370', 'hoarse', 'voice'],  
['102370', 'lab'],  
['102370', 'growth', 'cartilage', 'disorganized,', 'with', 'islands', 'of', 'cells',  
'and', 'abnormal'],  
['102370', 'collagen', 'arrangement'],...
```

A total of 166029 sentences taken from the OMIM database.
The initial numbers expresses OMIM entry codes.

GPT Capabilities

- Text generation
- Question answering
- Language translation
- Summarization
- Code generation
- ... audio, images

Let's train our own GPT model

```
>> python run.py preprocess
```

```
The corpus has 142 unique tokens.
```

```
SUCCESS
```

Let's train our own GPT model

```
>> python run.py train  
Initializing a new model.  
Parameters to be optimized: 4790926
```

10:10:27	step 0: train loss 4.9567, valid loss 4.9876
10:11:56	step 500: train loss 0.1477, valid loss 5.2037
10:13:49	step 1000: train loss 0.1291, valid loss 5.8455
10:15:12	step 1500: train loss 0.1191, valid loss 6.1366
10:16:35	step 2000: train loss 0.1167, valid loss 6.5854
10:17:58	step 2500: train loss 0.1106, valid loss 6.6639
10:19:20	step 3000: train loss 0.1105, valid loss 6.7045
10:20:47	step 3500: train loss 0.1099, valid loss 6.8777
10:22:07	step 4000: train loss 0.1095, valid loss 6.8912
10:23:28	step 4500: train loss 0.1105, valid loss 7.3837
10:24:50	step 4999: train loss 0.1085, valid loss 6.8775

Let's train our own GPT model

```
>> python run.py train --update
```

Let's train our own GPT model

```
>> python run.py chat
```


FLAIR Example 1: Tag Entities in Text

- Let's run **named entity recognition** (NER) over the following example sentence: *"I love Berlin and New York."*
- Our goal is to identify names in this sentence, and their types.
- To do this, all you need is to make a **Sentence** for this text, load a pre-trained model and use it to predict tags for the sentence:

```
from flair.data import Sentence
from flair.nn import Classifier

# make a sentence
sentence = Sentence('I love Berlin and New York.')
# load the NER tagger
tagger = Classifier.load('ner')
# run NER over sentence
tagger.predict(sentence)
# print the sentence with all annotations
print(sentence)
Sentence[7]: "I love Berlin and New York." → ["Berlin"/LOC, "New York"/LOC]
```

The output shows that both “Berlin” and “New York” were tagged as **location entities** (LOC) in this sentence.

FLAIR Example 2: Detect Sentiment

Let's run **sentiment analysis** over the same sentence to determine whether it is **POSITIVE** or **NEGATIVE**.

You can do this with essentially the same code as above. Just instead of loading the 'ner' model, you now load the 'sentiment' model:

```
from flair.data import Sentence
from flair.nn import Classifier

# make a sentence
sentence = Sentence('I love Berlin and New York.')
# load the sentiment tagger
tagger = Classifier.load('sentiment')
# run sentiment analysis over sentence
tagger.predict(sentence)
# print the sentence with all annotations
print(sentence)
Sentence[7]: "I love Berlin and New York." → POSITIVE (0.9982)
```

Tagging parts-of-speech

Syntax is fundamentally language-specific, so each language has different fine-grained parts-of-speech. Flair offers models for many languages:

... in English

```
from flair.nn import Classifier
from flair.data import Sentence

# load the model
tagger = Classifier.load('pos')
# make a sentence
sentence = Sentence('Dirk went to the store.')
# predict NER tags
tagger.predict(sentence)
# print sentence with predicted tags
print(sentence)
Sentence[6]: "Dirk went to the store." → ["Dirk"/NNP, "went"/VBD, "to"/IN, "the"/DT,
"store"/NN, "."/.]
```

“Dirk” is a proper noun (tag: NNP), and “went” is a past tense verb (tag: VBD).

... in German

```
from flair.nn import Classifier
from flair.data import Sentence

# load the model
tagger = Classifier.load('de-pos')
# make a sentence
sentence = Sentence('Dort hatte er einen Hut gekauft.')
# predict NER tags
tagger.predict(sentence)
# print sentence with predicted tags
print(sentence)
```

```
Sentence[7]: "Dort hatte er einen Hut gekauft." →
["Dort"/ADV, "hatte"/VAFIN, "er"/PPER, "einen"/ART,
"Hut"/NN, "gekauft"/VVPP, "."/$.]
```

... in Ukrainian

```
from flair.nn import Classifier
from flair.data import Sentence

# load the model
tagger = Classifier.load('pos-ukrainian')
# make a sentence
sentence = Sentence("Сьогодні в Знам'янці проживають  
нащадки поета — родина Шкоди.")
# predict NER tags
tagger.predict(sentence)
# print sentence with predicted tags
print(sentence)
```

... in Arabic

```
from flair.nn import Classifier
from flair.data import Sentence

# load the model
tagger = Classifier.load('ar-pos')
# make a sentence
sentence = Sentence('عمرو عادلي أستاذ للاقتصاد السياسي المساعد  
في الجامعة الأمريكية بالقاهرة')
# predict NER tags
tagger.predict(sentence)
# print sentence with predicted tags
print(sentence)
```

Tagging universal parts-of-speech (uPoS)

Universal parts-of-speech are a set of minimal syntactic units that exist across languages.

For instance, most languages will have VERBs or NOUNs.

FLAIR was trained over 14 languages to tag upos in **multilingual text**.

```
from flair.nn import Classifier
from flair.data import Sentence

# load model
tagger = Classifier.load('pos-multi')

# text with English and German sentences
sentence = Sentence('George Washington went to Washington. Dort kaufte er einen Hut.')
# predict PoS tags
tagger.predict(sentence)
# print sentence with predicted tags
print(sentence)
Sentence: "George Washington went to Washington . Dort kaufte er einen Hut ."
→ ["George"/PROPN, "Washington"/PROPN, "went"/VERB, "to"/ADP, "Washington"/PROPN, "."/PUNCT]
→ ["Dort"/ADV, "kaufte"/VERB, "er"/PRON, "einen"/DET, "Hut"/NOUN, "."/PUNCT]
```

List of POS Models

ID	Task	Language	Training Dataset	Accuracy	Contributor / Notes
‘pos’	POS-tagging	English	Ontonotes	98.19 (Accuracy)	
‘pos-fast’	POS-tagging	English	Ontonotes	98.1 (Accuracy)	(fast model)
‘upos’	POS-tagging (universal)	English	Ontonotes	98.6 (Accuracy)	
‘upos-fast’	POS-tagging (universal)	English	Ontonotes	98.47 (Accuracy)	(fast model)
‘pos-multi’	POS-tagging	Multilingual	UD Treebanks	96.41 (average acc.)	(12 languages)

List of POS Models

ID	Task	Language	Training Dataset	Accuracy	Contributor / Notes
'da-pos'	POS-tagging	Danish	Danish Dependency Treebank		AmaliePauli
'pt-pos-clinical'	POS-tagging	Portuguese	PUCPR	92.39	LucasFerroH AILab for clinical texts
' pos-ukrainian '	POS-tagging	Ukrainian	Ukrainian UD	97.93 (F1)	dchaplinsky

Classic Word Embeddings

Classic word embeddings are static and word-level.

- Each distinct word gets exactly one pre-computed embedding.
- Most embeddings fall under this class, including the popular GloVe or Komninos embeddings (e.g. Wikipedia).

```
from flair.embeddings import WordEmbeddings

# init embedding
glove_embedding = WordEmbeddings('glove')
# create sentence.
sentence = Sentence('The grass is green .')
# embed a sentence using glove.
glove_embedding.embed(sentence)
# now check out the embedded tokens.
for token in sentence:
    print(token)
    print(token.embedding)
```

This prints out the tokens and their embeddings. GloVe embeddings are Pytorch vectors of dimensionality 100.

ID	Language	Embedding
‘en-glove’ (or ‘glove’)	English	GloVe embeddings
‘en-extvec’ (or ‘extvec’)	English	Komninos embeddings
‘en-crawl’ (or ‘crawl’)	English	FastText embeddings over Web crawls
‘en-twitter’ (or ‘twitter’)	English	Twitter embeddings
‘en’ (or ‘en-news’ or ‘news’)	English	FastText embeddings over news and wikipedia data
‘de’	German	German FastText embeddings
‘fr’	French	French FastText embeddings
‘es’	Spanish	Spanish FastText embeddings

Custom embeddings

- Loading Gensim embeddings:

```
custom_embedding = WordEmbeddings('path/to/your/custom/embeddings.gensim')
```

- Converting FastText embeddings to Gensim:

```
import gensim
word_vectors = gensim.models.KeyedVectors.load_word2vec_format('/path/to/embeddings.txt', binary=False)
word_vectors.save('/path/to/converted')
```

- However, FastText embeddings have the functionality of returning vectors for out of vocabulary words using the sub-word information.



gensim

topic modelling for humans



Download

latest version from the Python Package Index



Direct install with:
easy_install -U gensim

Home

Tutorials

Install

Support

API

About

```
>>> from gensim import corpora, models, similarities
>>>
>>> # Load corpus iterator from a Matrix Market file on disk.
>>> corpus = corpora.MmCorpus('/path/to/corpus.mm')
>>>
>>> # Initialize Latent Semantic Indexing with 200 dimensions.
>>> lsi = models.LsiModel(corpus, num_topics=200)
>>>
>>> # Convert another corpus to the latent space and index it.
>>> index = similarities.MatrixSimilarity(lsi[another_corpus])
>>>
>>> # Compute similarity of a query vs. indexed documents
>>> sims = index[query]
```

Gensim is a FREE Python library



Scalable statistical semantics



Analyze plain-text documents for semantic structure



Retrieve semantically similar documents